

ORB-SLAM

1. **Visión general** del algoritmo original, tres threads que corren en paralelo (fuente 2016_SLAM_ORB_SLAM_TFM_PatriciaLópezTorres)

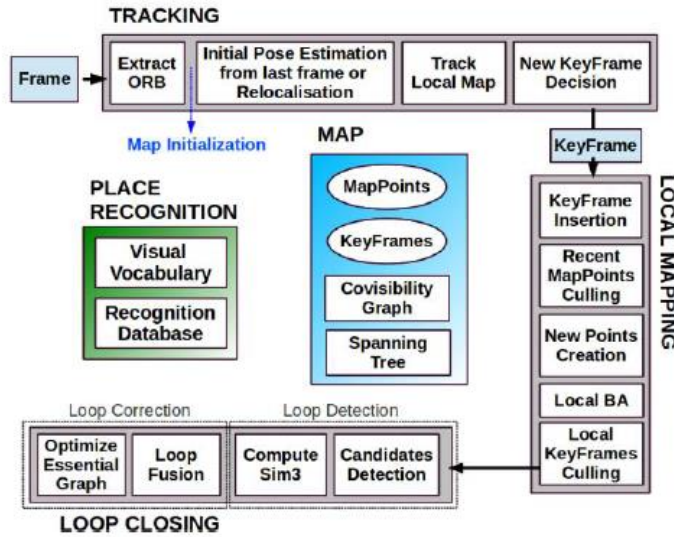


Figura 15: Módulos del algoritmo ORB-SLAM

2. **Descripción funcional del thread de tracking** (fuente: 2019_Liu_eSLAM)

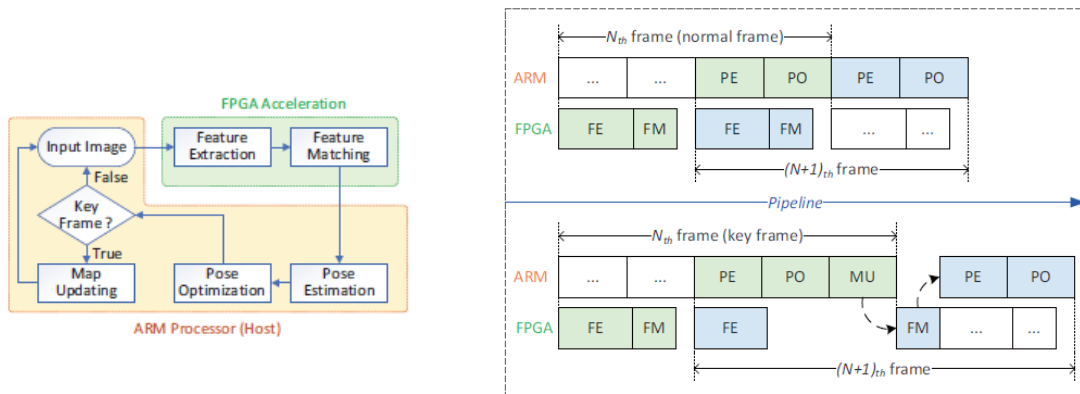


Figure 7: Parallelized pipeline of normal frame (upper) and key frame (lower), where FE refers to feature extraction, FM refers to feature matching, PE refers to pose estimation, PO refers to pose optimization and MU refers to map updating.

4. Descripción funcional de las etapas ORB extractor (fuente: 2022_Wasala_An Efficient Real-Time FPGA-Based ORB)

0. Resized to get several scaled images
1. Performing the fast accelerated segment test to determine the corners (FAST).
2. Filtration using non-maximum suppression (NMS).
3. Elimination of feature points for which it is not possible to determine the full context (31x31 pixels)
4. Filtering of feature points and leaving only the best N points.
5. Computation of the Harris score and re-filtering of feature points.
6. Calculation of the orientation of the feature point (intensity centroid).
7. Determination of the context 31 x 31 px and blurring with a Gaussian filter.
8. Determination of the binary feature descriptor (rBRIEF).

4.0 Resized

Se obtienen L imágenes con diferentes escalas a partir de la imagen original (scale ratio 1.2). A cada imagen se le pasa el ORB extractor

4.1 Corner detection (FAST)

Es un extractor de esquinas (corner detection) que serán los keypoints de la imagen. Como queremos obtener, a partir de él, un descriptor invariante a rotación, es necesario calcular la orientación del keypoint, obteniéndose el *oFAST* (oriented FAST). Para cada pixel se examina su vecindad (patch), 7x7 o 9x9, y se determina:

- a) **Keypoint detection.** Si es corner o no (valor binario) => se compara el valor del pixel con su vecindad. El pixel central con intensidad I_p se considera que es keypoint si existen n pixels contiguos (en una vecindad tipo Bresenham circle) que son más claros que $I_p + t$ o más oscuros que $I_p - t$ (t es un umbral, por defecto en OpenCV $t=10$). Los mejores resultados se obtienen para $n=9$ aunque otros autores utilizan $n=12$. (ver implementación en 2017_Sun)

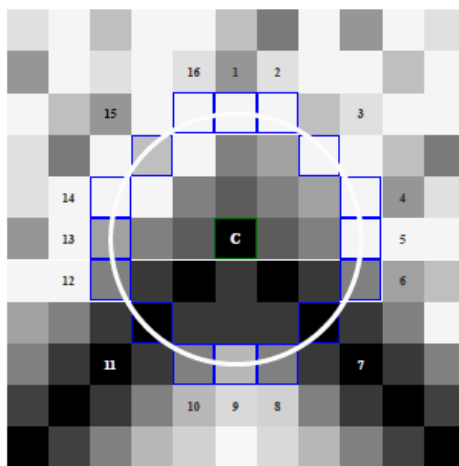


Figure 1. The Bresenham circle used in the FAST algorithm. Note that for $n = 9$ the centre point will be detected as a corner for a relatively wide range of possible thresholds t .

- b) **Keypoint score**, es un valor que indica la estabilidad del keypoint (ver 2022_Wasala), Para calcularlo se utiliza alguno de los siguientes scores:
 - *Harris score*: complejo de calcular por el número de operaciones MULT utilizadas

- *Sum of absolute differences* (SAD): es el mínimo entre la suma de diferencias en valor absoluto de los pixels bright y el pixel central, y lo mismo para los pixels dark.

$$V = \max \left(\sum_{x \in S_{bright}} |I_{p \rightarrow x} - I_p| - t, \sum_{x \in S_{dark}} |I_p - I_{p \rightarrow x}| - t \right)$$

$$S_{bright} = \{x | I_{p \rightarrow x} \geq I_p + t\}$$

$$S_{dark} = \{x | I_{p \rightarrow x} \leq I_p - t\}$$

- *FAST score* (openCV): una modificación del anterior. (i) para un conjunto de n pixels contiguos se calcula el valor mínimo de las dif en valor abs entre los pixels y el central (d_{min}^k), (ii) se repite para los 16 arcos de pixels contiguos (iii) el score es el máximo de estos.

$$d^k = \{|I_{p \rightarrow x} - I_p|, x \in \{1, \dots, 9\}\}$$

$$d_{min}^k = \min \{d_i^k, i \in \{1, \dots, 9\}\}$$

$$score = \max \{d_{min}^k, k \in \{1, \dots, 16\}\}$$

4.2 Non Maximum Supression (NMS)

Se eliminan los keypoints con menor valor del score (en una vecindad, 3 x 3)

4.3 Eliminación de keypoints para los que no se puede determinar el contexto (los pixels cercanos a los bordes)

4.4 Determinación de los N keypoints más relevantes. Se haría ordenando los keypoints por su score y quedándonos con los N mayores.

4.5 Cálculo de la orientación del keypoint. Se calcula la orientación (ángulo θ) del pixel respecto del centroide (centro de masas) del patch.

El cálculo de las coordenadas (x,y) del centro de masas (centroide):

$$CM_x = \text{Sum_xy} (x * I_p(x,y)) / \text{Sum_xy} (I_p(x,y)) = m_{10}$$

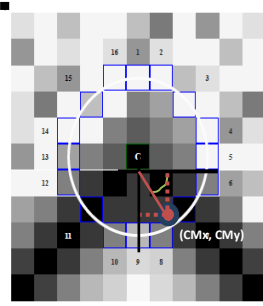
$$CM_y = \text{Sum_xy} (y * I_p(x,y)) / \text{Sum_xy} (I_p(x,y)) = m_{01}$$

Donde el ángulo puede calcularse como

$$\theta = \arctan (CM_y/CM_x) = \arctan (m_{01}/m_{10});$$

O de forma aproximada como:

$$\theta = \arctan2 (m_{01}, m_{10})$$



4.6 Determination of the context 31 x 31 px and blurring with a Gaussian filter.

Para cada keypoint se obtiene su contexto (patch 31 x 31) y se aplica un filtro de emborronamiento para reducir la influencia del ruido (filtro gaussiano)

4.7 Descriptor BRIEF representa el entorno local al keypoint. Utilizando el contexto del keypoint (patch) se calcula el descriptor: vector binario de 256 bits.

Para ello se elige de forma aleatoria 256 parejas de coordenadas $(x_u, y_u), (x_v, y_v)$ siguiendo una distribución gaussiana y se calcula el valor del test

$$\tau(\mathbf{p}; u_i, v_i) := \begin{cases} 1 & \text{for } I(u_i) < I(v_i) \\ 0 & \text{for } I(u_i) \geq I(v_i) \end{cases}$$

Con estos valores binarios se forma el descriptor (vector binario).

Para que el descriptor sea invariante a rotación, cada pareja de coordenadas se rota previamente un ángulo θ para alinearlas con la orientación del keypoint. Posteriormente se pasa el test. A este descriptor se le denomina **rBRIEF** (rotation aware BRIEF):

$$\begin{aligned} x' &= x \cdot \cos\theta - y \cdot \sin\theta \\ y' &= y \cdot \cos\theta + x \cdot \sin\theta \end{aligned}$$

Análisis

La mayoría de las etapas pueden migrarse a la FPGA sin incurrir en pérdidas de precisión notables respecto a la versión OpenCV, excepto 4.5 cálculo de la orientación y 4.7 cálculo del descriptor rBRIEF.

El cálculo de la orientación precisa de una operación de división y una función trigonométrica. La división puede implementarse, consumiendo bastantes recursos y ciclos, mientras que la forma más habitual de aproximar atan sería mediante CORDIC (multietapa y multiciclo). Por tanto esta traducción directa no permitiría el procesamiento en streaming.

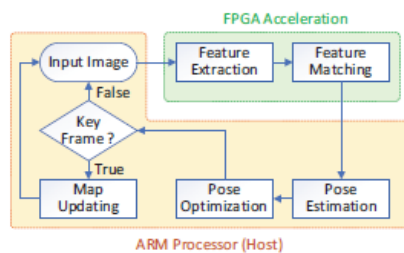
El cálculo de rBRIEF requiere de una rotación de las coordenadas, de nuevo se necesita CORDIC, además de consumir muchos recursos para conseguir procesamiento en streaming.

Se aconseja utilizar aproximaciones como las reportadas en el SoTA para estas dos etapas. Estas aproximaciones cambiarán los resultados respecto a los de openCV, por lo que será necesario evaluarlas previamente.

3. SoTA de implementaciones FPGA

3.1 eSLAM (Liu_2021)

Plantean la aceleración tanto de la Feature extraction como del Feature Matching, pero sin especificar como



Propone cambiar de orden las operaciones. El punto 4.4 filtrado de los N keypoints más relevantes pasa a ser la última operación. De esta forma la detección de los keypoints (FAST) y el cálculo del descriptor (BRIEF) pueden realizarse en streaming.

- Resized: 4-layer pyramid (sin detalles)
- Corner detection: FAST (7 x 7); Harris coner score (9 x 9);
- NMS (patch 3 x 3)
- Orientation: Gaussian (7x7) no calcula el ángulo sino el **intervalo de 11.25° en el que se encuentra el ángulo** (32 posibles intervalos entre 0° y 360°) .
- BRIEF: RS-BRIEF **Rotationally Symmetric BRIEF** evita tener que rotar los puntos antes del test
 - o se seleccionan 8 parejas alrededor del key point de acuerdo a una distribución gaussiana
 - o se rotan los puntos en incrementos de 11.25° y se obtienen las 256 parejas de puntos (las coordenadas se guardan en una tabla para acceder al valor de los

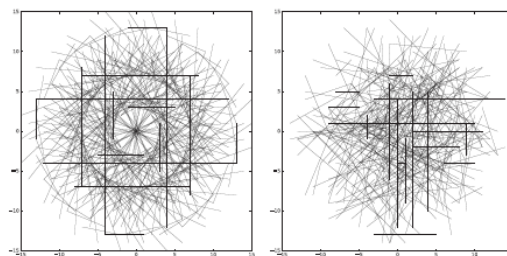


Figure 2: Pattern of RS-BRIEF (left) and BRIEF (right).

pixels)

Como consecuencia **no es necesario rotar las parejas de puntos antes de realizar el test. En lugar de calcular la orientación precisa del keypoint, esta se aproxima por el intervalo de 1.25° en el que se encuentra ese ángulo. De esa forma, se**

puede realizar el test sobre las 256 parejas de puntos sin rotar y posteriormente “rotar” el descriptor .

- El último paso es una ordenación de los keypoints en función para almacenar sólo los 1024 de score más alto. No se especifica como se implementa “a max-heap structure is utilized to guarantee that only the 1024 features with the best Harris scores are reserved”

3.1 Ac2SLAM 2021 (fuente: 2021_Wang_ac2SLAM_FPGA_Accelerated_High-Accuracy_SLAM_with_Heapsort_and_Parallel_Keypoint_Extractor)

Proponen la aceleración de la Feature extraction solamente. El resto queda para la CPU. Arquitectura tipo coprocesador.

Ofrece detalles de la implementación y código con licencia GPL3. También ofrece detalles de como optimizar las operaciones del Back-End

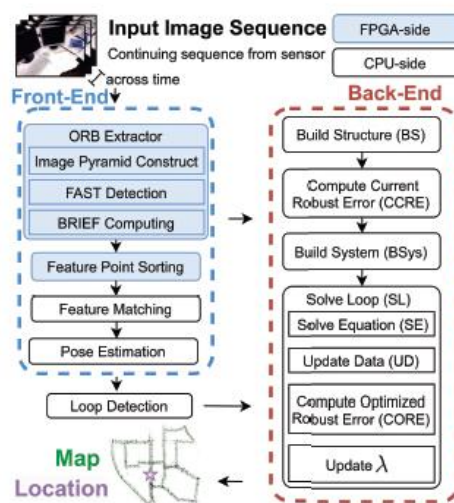


Fig. 1: Structure of the ORB-SLAM

Igual que en Liu_2019 se cambian de orden las operaciones el punto 4.4 filtrado de los N keypoints más relevantes pasa a ser la última operación. De esta forma la detección de los keypoints (FAST) y el cálculo del descriptor (BRIEF) pueden realizarse en streaming.

- Resized: bi-linear interpolation, 8 layers (scale ratio 1.2), de forma secuencial
IDEA: para 4 layers utiliza 4 imágenes consecutivas (la variación debería ser poca)
- Corner detection: FAST (9 x 9); coner score: oFAST (9 x 9);
- NMS (patch 9 x 9)
- Orientation: Gaussian (9x9) => (29 x 29), m10 (24b) m01 (24b),
angle=hls::atan2(m_01_f, m_10_f);
- BRIEF: RS-BRIEF **Rotationally Symmetric BRIEF** (descrito en Liu_2019)
 - o se seleccionan 8 parejas alrededor del key point de acuerdo a una distribución gaussiana
 - o se rotan los puntos en incrementos de 11.25° y se obtienen las 256 parejas de puntos (las coordenadas se guardan en una tabla para acceder al valor de los pixels)

Como consecuencia no es necesario rotar las parejas de puntos antes de realizar el test. En lugar de calcular la orientación precisa del keypoint, esta se aproxima por el intervalo de 1.25° en el que se encuentra ese ángulo. De esa forma, se puede realizar el test sobre las 256 parejas de puntos sin rotar y posteriormente “rotar” el descriptor .

- El último paso es una ordenación (con el algoritmo heapsort) de los keypoints en función de los scores para quedarse con un número fijo de keypoints.

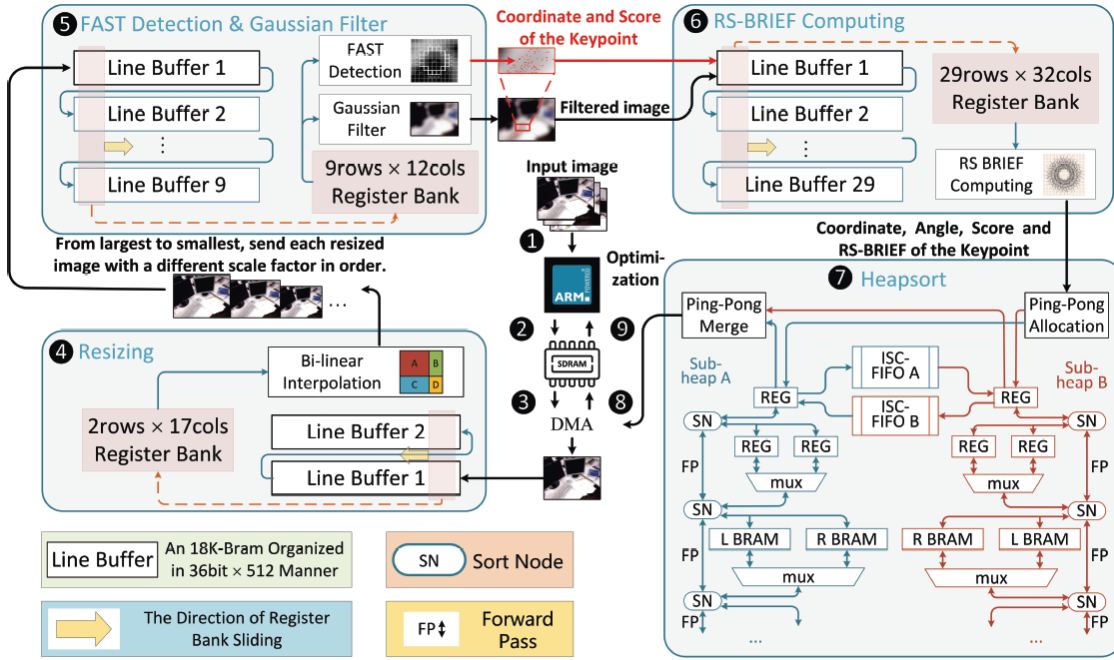


Fig. 2: Overall architecture of ac²SLAM

Comparado con eSLAM ofrece mejor rendimiento y accuracy que eSLAM a costa de un incremento moderado de los recursos

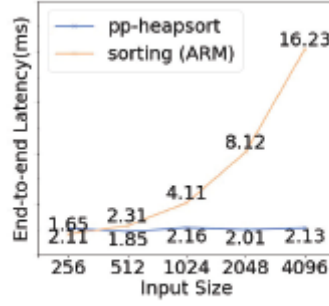
TABLE III: Utilization comparison of ac²SLAM and eSLAM

	LUT	FF	DSP	BRAM
ac²SLAM	146572	74166	173	61
eSLAM	56954	67809	111	78

TABLE IV: The Latency of Feature Extraction

	TUM (640 × 480)	KITTI(1241 × 376)
ac²SLAM	2.0 ms	5.1 ms
eSLAM	9.1 ms	not apply
ARM	80.2 ms	202.7 ms

IMPORTANTE: para un numero final de keypoints moderado 512/1024 podía suprimirse el módulo de Heapsort. Ver figura



(a) end-to-end latency.

3.2 Belmessoud 2022

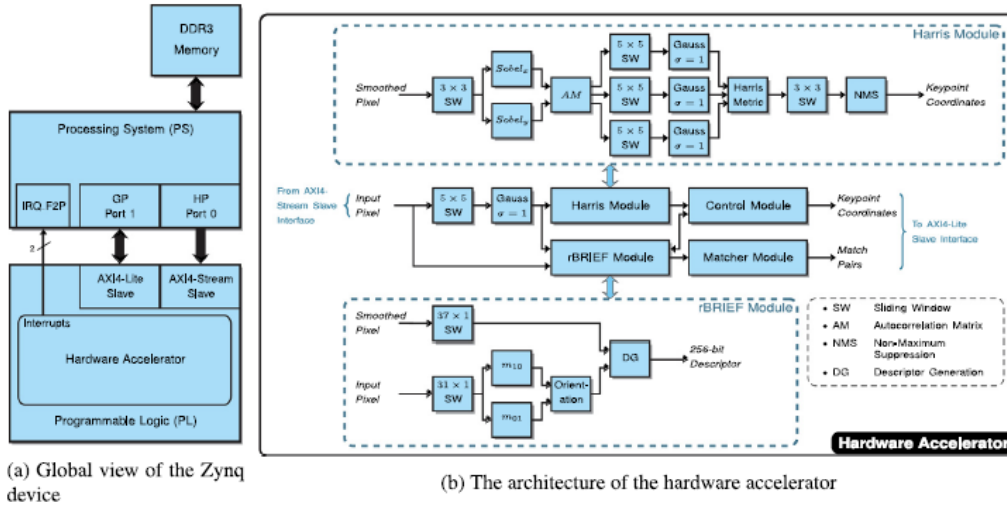


Fig. 2. Overview of the proposed architecture.

3.1 Wasala 2022

- Resized: bi-linear interpolation, 8 layers (scale ratio 1.2)
- Corner detection: FAST (9 x 9); coner score: oFAST (9 x 9);
- NMS (patch 9 x 9)
- Orientation: Gaussian (9x9) => (29 x 29), m10 (24b) m01 (24b), no computa el ángulo sino el **intervalo en el que se encuentra el ángulo** de entre 32 posibilidades (incrementos de 11,25°) . Utilizando el signo de m10 y m01 y sus valores puede determinarse el intervalo utilizando sólo 8 comparadores

$$\tan \theta_i \leq \tan \theta(x, y) \leq \tan \theta_{i+1}$$

$$m_{10}(x, y) \tan \theta_i \leq m_{01}(x, y) \leq m_{10}(x, y) \tan \theta_{i+1}$$

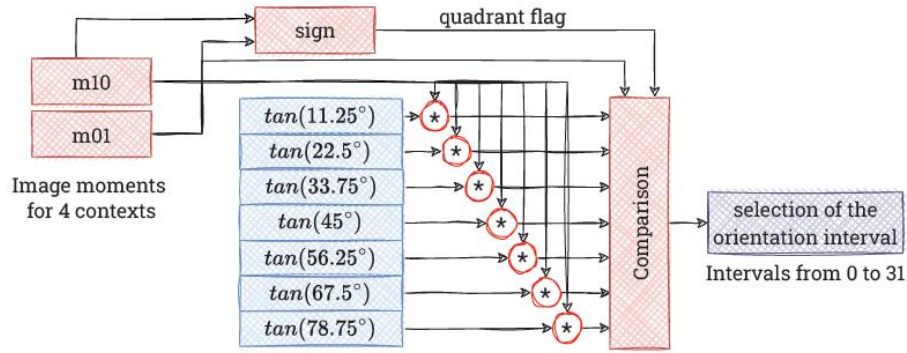
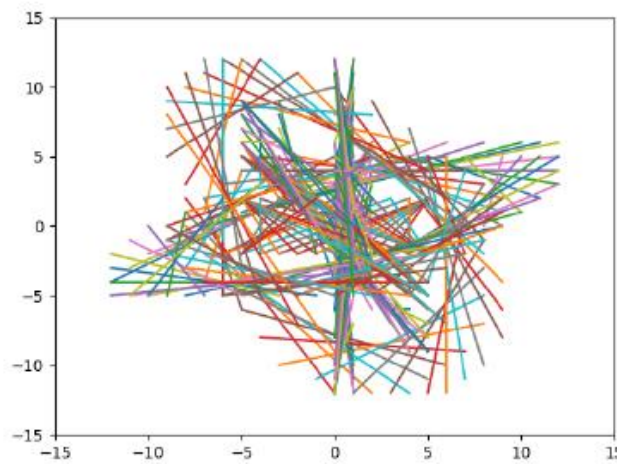


Figure 9. Scheme of the mechanism for determining the orientation by assigning a value to an appropriate interval.

y se rotan cada 11,25° obteniéndose 256 parejas. En el trabajo original de Calonder se prueban varias formas de seleccionar las parejas. La forma que mejor resultado ofrece

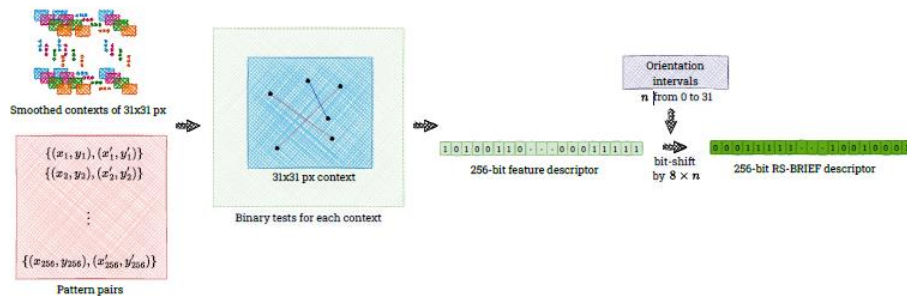


es utilizar seleccionarlas aleatoriamente de una distribución gaussiana.

Se leen las parejas y se realiza el test

$$\tau(\mathbf{p}; u_i, v_i) := \begin{cases} 1 & \text{for } I(u_i) < I(v_i) \\ 0 & \text{for } I(u_i) \geq I(v_i) \end{cases}$$

Al vector de 256bits se le aplica un desplazamiento de $8 \times n$ bits en función del ángulo (orientación del keypoint) obtenido en el anterior paso (n = intervalo del ángulo; 0..31)



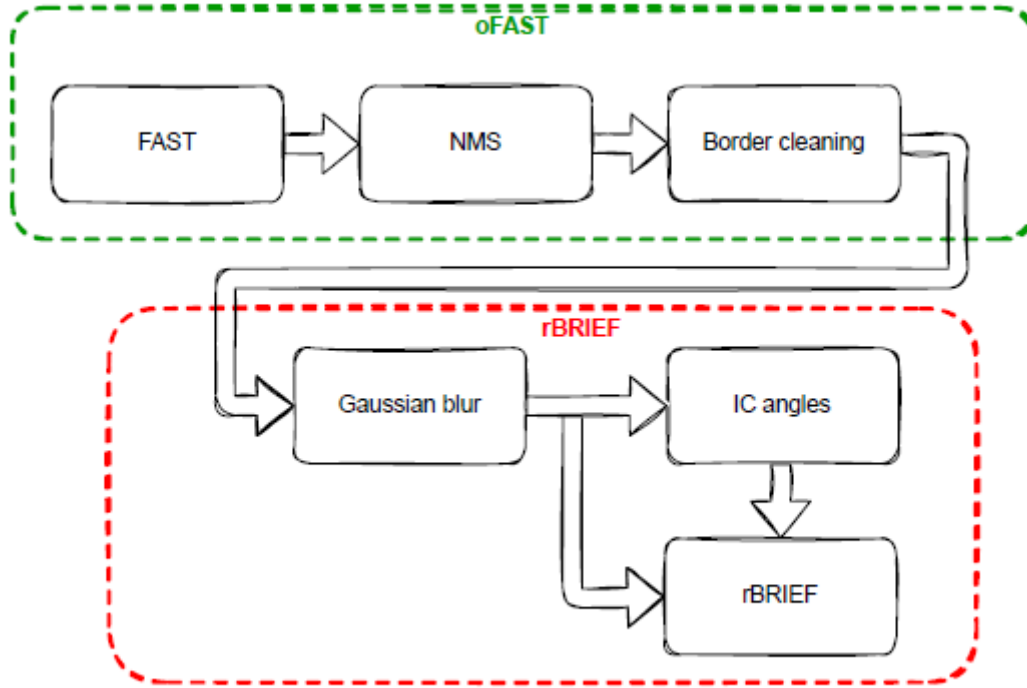


Figure 2. Overall scheme of our hardware implementation of the ORB algorithm on a SoC FPGA platform.

Comparativa

Table 3. Comparison of our proposed solution with other algorithms reported in scientific papers. A direct comparison of our implementation can only be made with works that also use video pass-through architectures, i.e., [13,14,16,22–24]. In comparison to previous works, we use noticeably more FFs and DSP blocks. This is due to the 4 ppc real-time implementation of the BRIEF descriptor.

	FPGA	Algorithm	# of LUTs	# of Registers, FFs	BRAM	DSP Blocks	FPS	Freq. [MHz]	Resolution
[25]	AMD Xilinx Zynq-7000	ORB ^{†,1}	4257	3187	576 Kb	-	55	100	640 × 480
[13]	AMD Xilinx ZedBoard	ORB [#]	9866	17,412	1.33 Mb	-	325	100	640 × 480
[14]	Altera Stratix V	ORB ^{#,4}	25,648	21,791	9.44 Mb	8	67	203	640 × 480
[26]	Altera Aria V	ORB ^{†,8}	206,000	231,973	8.58 Mb	449	72	150	1920 × 1080
[27]	AMD Xilinx Kintex-7	ORB [†]	80,472	112,166	35 Kb	0	310	100	512 × 512
[16]	AMD Xilinx ZedBoard	FAST ^{‡,2,6}	5700	6272	1.984 Mb	-	63	148.5	1920 × 1080
[22]	Altera Aria V	BRIEF ^{‡,3}	12,523	10,019	110 Kb	0	60	175	1920 × 1080
[12]	AMD Xilinx Ultrascale+	ORB ^{†,5}	28,168	9528	1.47 Mb	33	108	200	1920 × 1080
[9]	AMD Xilinx XCZ7045	ORB ^{†,6}	56,954	67,809	2.73 Mb	111	76	100	640 × 480
[23]	AMD Xilinx Kintex-7	ORB [#]	54,435	30,281	1.836 Mb	44	161	150	1280 × 720
[28]	Altera Cyclone V	ORB ^{†,9}	5711	5453	0.3–2.3 Mb	-	325	100	1280 × 720
[24]	AMD Xilinx Virtex-7	ORB ^{‡,6,9}	71,423	49,649	3.132 Mb	285	68.8	142.8	1920 × 1080
[18]	AMD Xilinx ZCU 104	ORB ^{†,6,7}	146,572	74,166	7.43 Mb	173	-	100	-
Ours	AMD Xilinx ZCU 104	ORB [#]	100,606	140,291	6.7 Mb	683	60	150	3840 × 2160

[‡] Stream-based architecture. [†] Non-stream-based architecture. Image is stored in external memory. ¹ This work is not rotation-invariant. ² Only oFAST module is implemented. ³ Only rBRIEF module is implemented. ⁴ This work uses a 2-level image pyramid. ⁵ This work uses a 3-level image pyramid. ⁶ This work uses a 4-level image pyramid. ⁷ This work uses an 8-level image pyramid. ⁸ This work uses a 9-level image pyramid. ⁹ Matching module is also implemented.